实验二　进程的创建

实验目的

1、掌握进程的概念，明确进程的含义

2、认识并了解并发执行的实质

实验内容

1、编写一段程序，使用系统调用 fork( )创建两个子进程。当此程序运行时，在系统中有一个父进程和两个子进程活动。让每一个进程在屏幕上显示一个字符：父进程显示'a'，子进程分别显示字符'b'和字符'c'。试观察记录屏幕上的显示结果，并分析原因。

2、修改上述程序，每一个进程循环显示一句话。子进程显示'daughter　…'及'son　……'，父进程显示 'parent　……'，观察结果，分析原因。

实验指导

一、 进程

LINUX 作为一个多用户操作系统，支持多道程序设计，支持分时处理和软中断处理，也带有某些微内核的特征。LINUX 进程控制块是 LINUX 最复杂的数据结构之一，内容包括进程调度、信号处理、进程队列指针、进程标识和用户标识、定时控制、信号量处理、上下文切换、文件系统管理、内存管理、进程状态和进程标志等信息，总计有 80 多类属性。LINUX 进程控制块常驻内存。LINUX 进程控制块由 struct task_struct 定义，长 1680 个字节。见附录二。

进程是一个动态的概念。LINUX 的进程状态有：

TASK_RUNNING　　正在运行或在就绪队列中准备运行的进程

TASK_INTERRUPTIBLE　处于等待队列中的进程，待资源有效时唤醒，也可由其它进程通过信号或定时中断唤醒

TASK_UNINTERRUPTIBLE 处于等待队列中的进程，待资源有效时唤醒，不可由其它进程通过信号或定时中断唤醒

TASK_ZOMBIE　　进程结束但尚未消亡释放 PCB，（僵死状态）

TASK_STOPPED　　进程被暂停，通过其它进程信号才能唤醒

各进程状态之间的转换关系见图 1。

二、所涉及的系统调用

1、fork( )

创建一个新进程。

系统调用格式：　　pid=fork( )

参数定义：　　int　fork( )

fork( )返回值意义如下：若子进程创建失败，则调用进程获得返回值-1。若创建成功，在子进程中，fork( )返回值为 0，表示当前进程是子进程；在父进程中，fork( )返回值为子进程的 id 值（>0）。

如果 fork( )调用成功，它向父进程返回子进程的 PID，并向子进程返回 0，即 fork( )调用一次，但在不同进程返回不同的值。此时 OS 在内存中建立一个新进程，所建的新进程是调用 fork( )父进程（parent process）的副本，称为子进程（child process）。子进程继承了父进程的许多特性，并具有与父进程完全相同的用户级上下文。父进程与子进程并发执行。

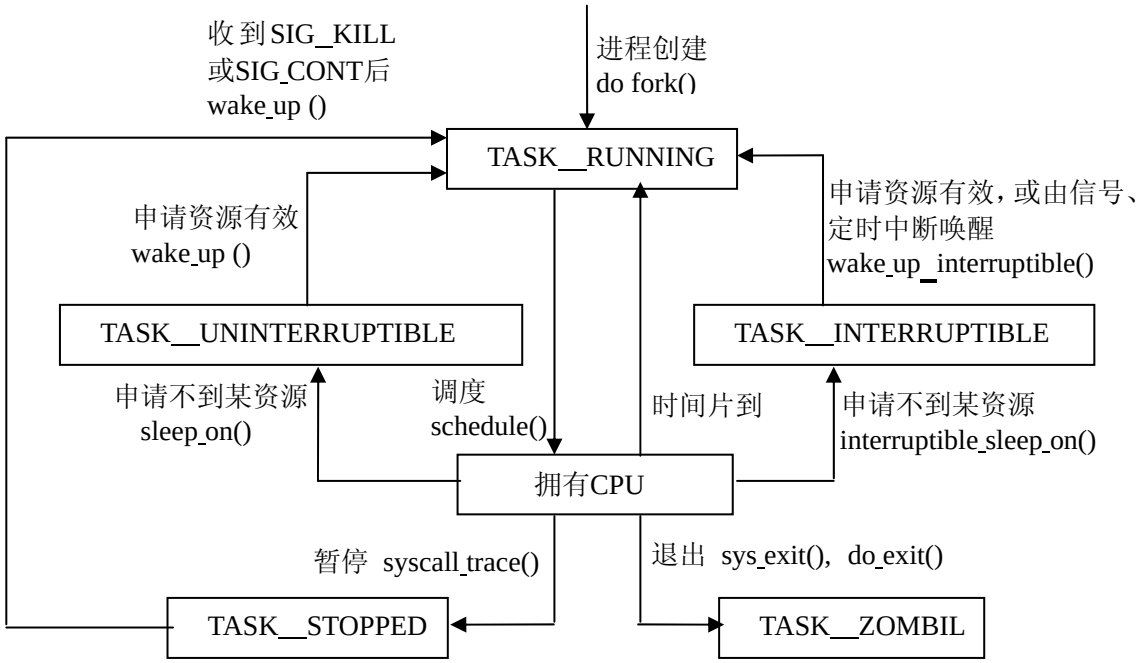


图 1　LINUX 进程状态之间的转换

核心为 fork( )完成以下操作：

（1）为新进程分配一进程表项和进程标识符

进入 fork( )后，核心检查系统是否有足够的资源来建立一个新进程。若资源不足，则 fork( )系统调用失败；否则，核心为新进程分配一进程表项和唯一的进程标识符。

（2）检查同时运行的进程数目

超过预先规定的最大数目时，fork( )系统调用失败。

（3）拷贝进程表项中的数据

将父进程的当前目录和所有已打开的数据拷贝到子进程表项中，并置进程的状态为“创建”状态。

（4）子进程继承父进程的所有文件

对父进程当前目录和所有已打开的文件表项中的引用计数加 1。

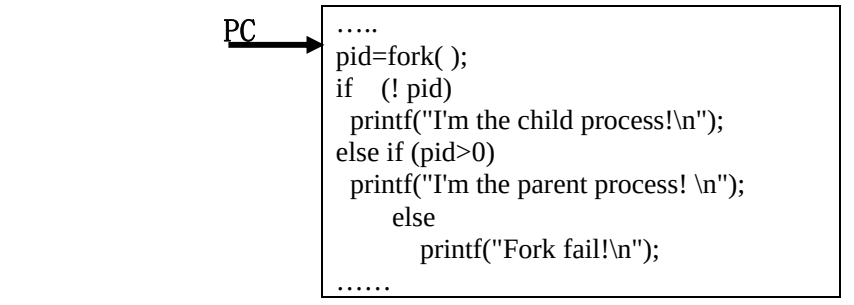
（5）为子进程创建进程上、下文

进程创建结束，设子进程状态为“内存中就绪”并返回子进程的标识符。

（6）子进程执行

虽然父进程与子进程程序完全相同，但每个进程都有自己的程序计数器 **PC(注意子进程的 PC 开始位置)**，然后根据 pid 变量保存的 fork( )返回值的不同，执行了不同的分支语句。

例：



2．getpid( )，getppid( )

getpid( )　　获得当前进程的进程 id

getppid( )　　获得当前父进程的进程 id

调用方式：

```
#include <sys/types.h>
#include <unistd.h>
pid_t getpid(void);
pid_t getppid(void);
```

四、参考程序

1、

```
#include <stdio.h>
main( )
{
    int p1,p2;
    while((p1=fork( ))= = -1);          /*创建子进程 p1*/
    if (p1= =0)   putchar('b');
    else
        {
            while((p2=fork( ))= = -1);    /*创建子进程 p2*/
            if(p2= =0)   putchar('c');
            else   putchar('a');
        }
}
```

2、

```
#include <stdio.h>
main( )
```

```
{
    int p1,p2,i;
    while((p1=fork( ))= = -1);                  /*创建子进程 p1*/
    if (p1= =0)
        for(i=0;i<10;i++)
            printf("daughter    %d\n",i);
    else
        {
            while((p2=fork( ))= = -1);      /*创建子进程 p2*/
            if(p2= =0)
                for(i=0;i<10;i++)
                    printf("son    %d\n",i);
            else
                for(i=0;i<10;i++)
                    printf("parent    %d\n",i);
        }
}
```

五、运行结果

1、bca，bac, abc ,……都有可能。

2、 parent…
son…
daughter..
daughter..

或 parent…
son…
parent…
daughter…等

六、分析原因

除 strace 外，也可用 ltrace -f -i -S ./executable-file-name 查看以上程序执行过程。

1、从进程并发执行来看，各种情况都有可能。上面的三个进程没有同步措施，所以父进程与子进程的输出内容会叠加在一起。输出次序带有随机性。Fork()创建进程所需的时间多于输入一个字符的时间，因此在主进程创建进程 2 的同时，进程 1 就输出了 b，而进程 2 和主程序的输出次序是随机性的。

2、由于函数 printf( )在输出字符串时不会被中断，因此，字符串内部字符顺序输出不变。但由于进程并发执行的调度顺序和父子进程抢占处理机问题，输出字符串的顺序和先后随着执行的不同而发生变化。这与打印单字符的结果相同。

补充：进程树

在 UNIX 系统中，只有 0 进程是在系统引导时被创建的，在系统初启时由 0 进程创建 1 进程，以后 0 进程变成对换进程，1 进程成为系统中的始祖进程。UNIX 利用 fork( )为每个终端创建一个子进程为用户服务，如等待用户登录、执行 SHELL 命令解释程序等，每个终端进程又可利用 fork( )来创建其子进程，从而形成一棵进程树。可以说，系统中除 0 进程外的所有进程都是用 fork( )创建的。

七、思考题

（1）系统是怎样创建进程的？

（2）当首次调用新创建进程时，其入口在哪里？

实验二　进程的控制

实验目的

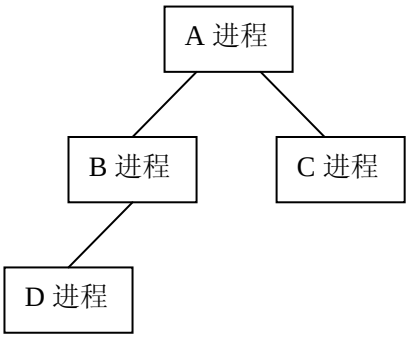
1、掌握进程另外的创建方法

2、熟悉进程的睡眠、同步、撤消等进程控制方法

实验内容

1、用 fork( )创建一个进程，再调用 exec( )用新的程序替换该子进程的内容

2、用 fork( )建立如下形式的进程树：



各个进程中都打印出本身 PID 和其父进程的 PID，并用 wait( )来控制进程执行顺序，打印出正确和期望的结果。

实验指导

一、所涉及的系统调用

在 UNIX/LINUX 中 fork( )是一个非常有用的系统调用，但在 UNIX/LINUX 中建立进程除了 fork( )之外，也可用与 fork( ) 配合使用的 exec( )。

1、exec( )系列系统调用

系统调用 exec( )系列，也可用于新程序的运行。fork( )只是将父进程的用户级上下文拷贝到新进程中，而 exec( )系列可以将一个可执行的二进制文件覆盖在新进程的用户级上下文的存储空间上，以更改新进程的用户级上下文。exec( )系列中的系统调用都完成相同的功能，它们把一个新程序装入内存，来改变调用进程的执行代码，从而形成新进程。如果 exec( )调用成功，调用进程将被**覆盖**，然后从新程序的入口开始执行，这样就产生了一个新进程，新进程的进程标识符 id 与调用进程相同。

exec( )没有建立一个与调用进程并发的子进程，而是用新进程取代了原来进程。所以 exec( )调用成功后，没有任何数据返回，这与 fork( )不同。exec( )系列系统调用在系统库 unistd.h 中，共有 execl、execlp、execle、execv、execvp 五个，其基本功能相同，只是以不同的方式来给出参数。

一种是直接给出参数的指针，如：

```
int    execl(path,arg0[,arg1,...argn],0);
char    *path,*arg0,*arg1,...,*argn;
```

另一种是给出指向参数表的指针，如：

```
int execv(path,argv);
char *path,*argv[ ];
```

具体使用可参考有关书。

2、exec( )和 fork( )联合使用

系统调用 exec 和 fork( )联合使用能为程序开发提供有力支持。用 fork( )建立子进程，然后在子进程中使用 exec( )，这样就实现了父进程与一个与它完全不同子进程的并发执行。

一般，wait、exec 联合使用的模型为：

```
int status;
   .............
if (fork( )= =0)
   {
        ...........;
        execl(...);
        ...........;
   }
wait(&status);
```

3、wait（）

等待子进程运行结束。如果子进程没有完成，父进程一直等待。wait( )将调用进程挂起，直至其子进程因暂停或终止而发来软中断信号为止。如果在 wait( )前已有子进程暂停或终止，则调用进程做适当处理后便返回。

系统调用格式：

```
#include <sys/types.h>
#include <sys/wait.h>
pid_t    wait(int *status)
```

其中，status 是用户空间的地址。它的低 8 位反映子进程退出状态，为 0 表示子进程正常结束，非 0 则表示出现了各种各样的问题；高 8 位则带回了 exit( )的返回值。exit( )返回值由系统给出。wait 的返回值为结束子进程的 PID。

核心对 wait( )作以下处理：

（1）首先查找调用进程是否有子进程，若无，则返回出错码；

（2）若找到一处于“僵死状态”的子进程，则将子进程的执行时间加到父进程的执行时间上，并释放子进程的进程表项；

（3）若未找到处于“僵死状态”的子进程，则调用进程便在可被中断的优先级上睡眠，等待其子进程发来软中断信号时被唤醒。

4、exit（）

终止进程的执行。

系统调用格式：

```
        void exit(int status)
```

其中，status 是返回给父进程的一个整数，以备查考。

为了及时回收进程所占用的资源并减少父进程的干预，UNIX/LINUX 利用 exit( )来实现进程的自我终止，通常父进程在创建子进程时，应在进程的末尾安排一条 exit( )，使子进程自我终止。exit(0)表示进程正常终止，exit(1)表示进程运行有错，异常终止。

如果调用进程在执行 exit( )时，其父进程正在等待它的终止，则父进程可立即得到其返回的整数。核心须为 exit( )完成以下操作：

（1）关闭软中断

（2）回收资源

（3）写记帐信息

（4）置进程为“僵死状态”

二、参考程序

```
#include<stdio.h>
#include<unistd.h>
main( )
```

```
{
    int pid;
    pid=fork( );              /*创建子进程*/
    switch(pid)
        {
            case  -1:                                   /*创建失败*/
                printf("fork fail!\n");
                exit(1);
            case  0:                                      /*子进程*/
                execl("/bin/ls","ls","-1","-color",NULL);
                printf("exec fail!\n");
                exit(1);
            default:                                      /*父进程*/
                wait(NULL);                             /*同步*/
                printf("ls completed !\n");
                exit(0);
        }
}
```

三、运行结果

执行命令 ls -l -color ，（按倒序）列出当前目录下所有文件和子目录；

ls completed!

四、分析原因

程序在调用 fork( )建立一个子进程后，马上调用 wait( )，使父进程在子进程结束之前，一直处于睡眠状态。子进程用 exec( )装入命令 ls ，exec( )后，子进程的代码被 ls 的代码取代，这时子进程的 PC 指向 ls 的第 1 条语句，开始执行 ls 的命令代码。

注意在这里 wait( )给我们提供了一种实现进程同步的简单方法。

五、思考

（1）可执行文件加载时进行了哪些处理？

（2）什么是进程同步？wait( )是如何实现进程同步的？

（3）为什么有时打印出父进程的 PID 不正确，是 1？